Introduction to Object-Oriented Programming (OOP) in C#

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects rather than functions and logic**. In OOP, programs are structured using **classes and objects**, which represent real-world entities.

C# is a **fully object-oriented language**, meaning most of the features in C# are designed around OOP principles.

---

1. What is OOP?

Object-Oriented Programming (OOP) is a programming approach where software is built using **objects that interact with each other**.

An **object** represents a **real-world entity** with:

- **Properties (Data / State)**
- **Methods (Behavior / Actions)**

Real-world Example

Consider a **Car**.

Real World	**In OOP**
Car	Class
Color, Model, Speed	Properties
Start(), Stop(), Accelerate()	Methods
A specific car (Honda City)	Object

---

2. Why OOP is Important

OOP helps developers build **scalable, maintainable, and reusable applications**.

Advantages

1. **Code Reusability**
2. **Better Modularity**

3. **Easy Maintenance**
4. **Improved Security**
5. **Better Real-world Mapping**

---

3. Basic Terminology in OOP

Before understanding OOP principles, we must understand these key terms.

3.1 Class

A **class** is a blueprint or template used to create objects.

It defines:

- Variables (fields)
- Methods (functions)
- Properties

Example

```
class Student
{
   public int Id;
   public string Name;

   public void Display()
   {
      Console.WriteLine(Id + " " + Name);
   }
}
```

Here:

- **Student** → Class
- **Id, Name** → Variables

- **Display()** → Method

3.2 Object

An **object** is an instance of a class.

A class defines the structure, but an object **represents a real entity in memory**.

Example

```
Student s1 = new Student();

s1.Id = 1;

s1.Name = "Rahul";
```

---

4. Four Pillars of OOP

OOP in C# is built on **four fundamental principles**.

1. **Encapsulation**
2. **Inheritance**
3. **Polymorphism**
4. **Abstraction**

4.1 Encapsulation

Encapsulation means **binding data and methods together into a single unit** and **restricting direct access to data**.

It protects the data from unauthorized access.

Example

```
class BankAccount

{

   private double balance;
```

```
    public void Deposit(double amount)
    {
        balance += amount;
    }

    public double GetBalance()
    {
        return balance;
    }
}
```

Usage

```
BankAccount account = new BankAccount();

account.Deposit(5000);

Console.WriteLine(account.GetBalance());
```

Key Idea

Data + Methods = Single Unit

Encapsulation is implemented using:

- Access Modifiers
- Properties
- Methods

4.2 Inheritance

Inheritance allows one class to **reuse properties and methods of another class**.

4.3 Polymorphism

Polymorphism means **"many forms"**.

A single method can perform **different tasks depending on context**.

There are **two types**:

1. **Compile-Time Polymorphism (Method Overloading)**
2. **Run-Time Polymorphism (Method Overriding)**

4.4 Abstraction

Abstraction means **hiding implementation details and showing only essential features**.

In C#, abstraction is achieved using:

1. **Abstract Classes**
2. **Interfaces**

---

7. OOP vs Procedural Programming

Feature	**Procedural**	**OOP**
Focus	Functions	Objects
Data Security	Less	High
Reusability	Limited	High
Code Structure	Linear	Modular

Access Modifiers in C# (OOP Concept)

Access Modifiers control the **visibility and accessibility** of classes, methods, variables, and properties.

They are used to **protect data and restrict access** from outside the class or assembly. This concept is closely related to **Encapsulation** in Object-Oriented Programming.

In C#, access modifiers define **who can access a member** and **from where it can be accessed**.

---

1. Why Access Modifiers are Important

Access modifiers help in:

- **Data protection**
- **Encapsulation**
- **Controlling code access**
- **Preventing accidental modifications**
- **Improving maintainability**

Example scenario:

In a **Banking Application**, the **Account Balance** should not be modified directly by other classes.

Instead, access should be controlled through methods like:

- Deposit()
- Withdraw()

This is achieved using **access modifiers**.

---

2. Types of Access Modifiers in C#

C# provides the following access modifiers:

Access Modifier	Accessible Within Class	Same Project	Derived Class	Outside Assembly
public	Yes	Yes	Yes	Yes
private	Yes	No	No	No
protected	Yes	No	Yes	No
internal	Yes	Yes	Yes	No

Access Modifier	Accessible Within Class	Same Project	Derived Class	Outside Assembly
protected internal	Yes	Yes	Yes	Yes (derived class)
private protected	Yes	Yes (derived only)	Yes	No

✅ **Summary**

OOP is a programming paradigm based on:

1. **Encapsulation** – Protect data
2. **Inheritance** – Reuse code
3. **Polymorphism** – Multiple behaviors
4. **Abstraction** – Hide complexity

These principles help build **large, maintainable, and scalable applications in C# and .NET**.